

Exercise #2 - Edge Detection using Cameras and Digital Video

Mason McGaffin

Exercise #2

ECEN 5763 Embedded Computer Vision

University of Colorado Boulder

June 19, 2026

Contents

1 BIG BUCK BUNNY	3
2 SOBEL EDGE DETECTION	4
3 CANNY EDGE DETECTION	5
4 COMPARE OPENCV AND USB WEBCAM	7
4.1 COMPUTER_VISION_CV4_TESTED/SIMPLER-CAPTURE-4/CAPTURE.....	7
4.2 SIMPLE-CAPTURE/CAPTURE	9
5 INTERACTIVE EDGE DETECTION	12
6 REFERENCES.....	16
7 APPENDIX	17

1 Big Buck Bunny

Verify that your ffmpeg installation on JetPack Linux is working (already installed on Jetson Nano 2g) by taking a video from Big Buck Bunny (Lab2-Examples/) and converting the first 100 frames into PPM or JPEG individual frames. Paste the 100th frame from Big Buck Bunny into your report (read [FFMPEG FAQ](#), and class notes on FFMPEG).

Video File: [Big_Buck_Bunny_720p.mp4](#)

The original way I did this, without consulting the internet, was running Command 1 to generate the first 100 frames and then selecting the 100th frame for the report. I knew there had to be a way to discard the unneeded frames and speed up the ffmpeg application execution; this led to Command 2. I ran a diff command on both of the frames generated and there was no difference between the files, indicating the commands generated the exact same 100th frame as expected.

Command 1:

```
ffmpeg -i Big_Buck_Bunny_720p.mp4 -vframes 100 first_100_frames/frame_%03d.ppm
```

Command 2:

```
ffmpeg -i Big_Buck_Bunny_720p.mp4 -vf "select=eq(n\,99)" -vframes 1 frame_100.ppm
```

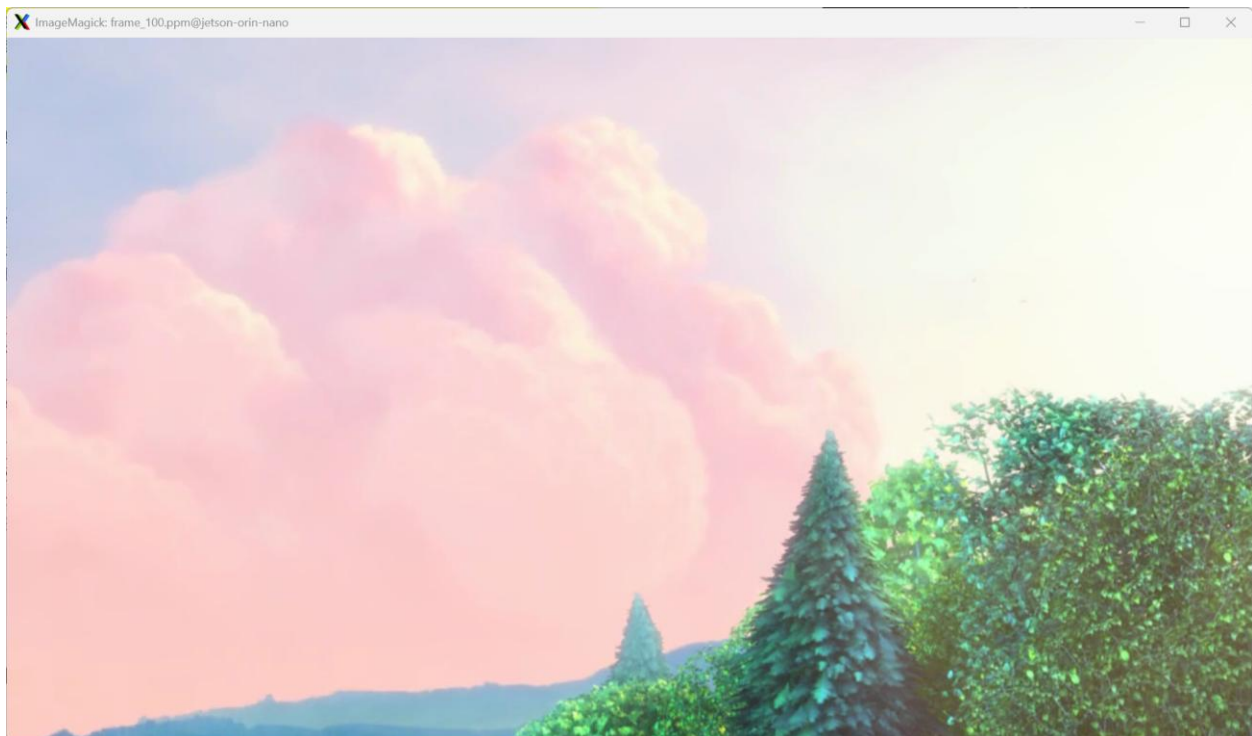


Figure 1.1 100th Frame of Big_Buck_Bunny_720p.mp4

2 Sobel Edge Detection

Using ffmpeg on your Jetson, take the 100th frame from Big Buck Bunny or the 400th frame from Open Source HD (your choice) and apply Sobel Edge Detection to it using with OpenCV – put your transformed frame into your report and describe any options you set or used. Explain the concept behind the Sobel operator as best you can.

The Sobel operator is a edge detection method that uses pixel brightness to determine edges. The operator uses an intensity gradient to approximate how rapidly pixel brightness changes. The operator first computes the horizontal and vertical gradients of the image. Then, for each pixel, it calculates the magnitude and direction of these gradients. A greater intensity gradient indicates stronger edges. The direction gives the edge orientation. This operator has very low computational cost and is easy to implement, making it excellent for simple edge detection applications that have low latency, real-time requirements.

I used [this source code](#) to apply Sobel edge detection to the image in Figure 1.1. I modified the code to include options to increase and decrease scale, ksize, and delta, so the user had more options to interact with the application. I also included logging that reflected the updated configuration after the user had pressed a key to make changes.

After playing with the application for a little bit, I decided that this configuration looked cool. Settings: scale = -15, ksize = 3, delta = 50.

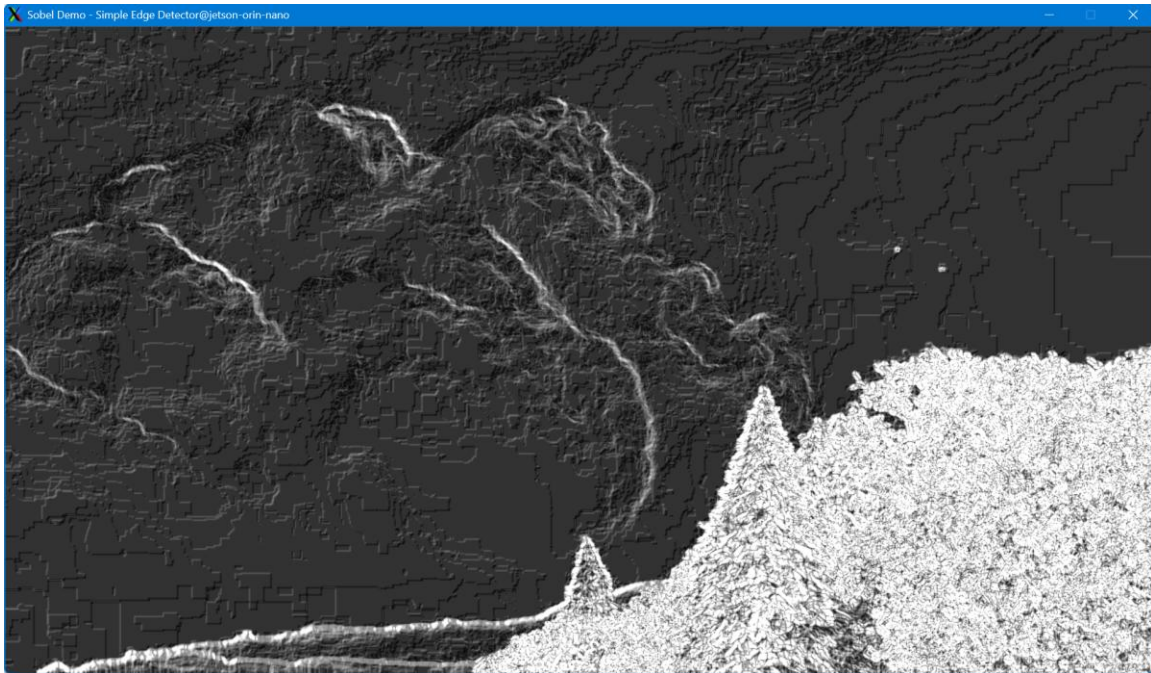


Figure 2.1 Sobel Edge Detection 100th Frame of Big_Buck_Bunny_720p.mp4

3 Canny Edge Detection

Using ffmpeg on your Jetson, take the 200th frame from Big Buck Bunny or the 800th frame from Open Source HD and apply Canny Edge Detection to it using OpenCV – put your transformed frame into your report and describe any options you set or used. Explain the concept behind the Canny edge detector as best you can.

The Canny edge detector is a multi-stage operator that requires higher compute time than the Sobel operator. It was designed to improve edge detection and localization and reduce false edges. First, the operator applies a Gaussian Blur to smooth the image which reduces noise that could cause false edges. Next, it computes a gradient similar to Sobel. Then, the operator applies a suppression algorithm that keeps only local maxima resulting in thin, one-pixel width edges. Next, the minimum low threshold is applied, as configured by the user, to specify how strong an edge must be to be kept. Finally, edge tracking occurs which keeps some weaker edges if they are attached to strong edges. Although the compute cost is higher, and this edge detection method is not as fast for real-time systems, it has significantly improved accuracy and quality making it great for object detection and high-quality image processing applications.

Command:

```
ffmpeg -i Big_Buck_Bunny_720p.mp4 -vf "select=eq(n\,199)" -vframes 1 frame_200.ppm
```

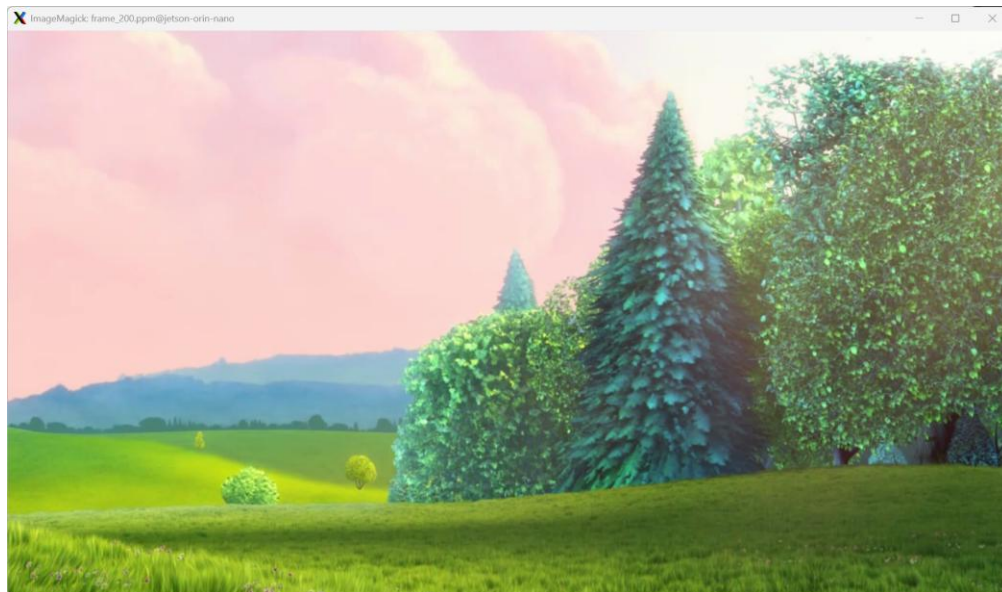


Figure 3.1 200th Frame of Big_Buck_Bunny_720p.mp4

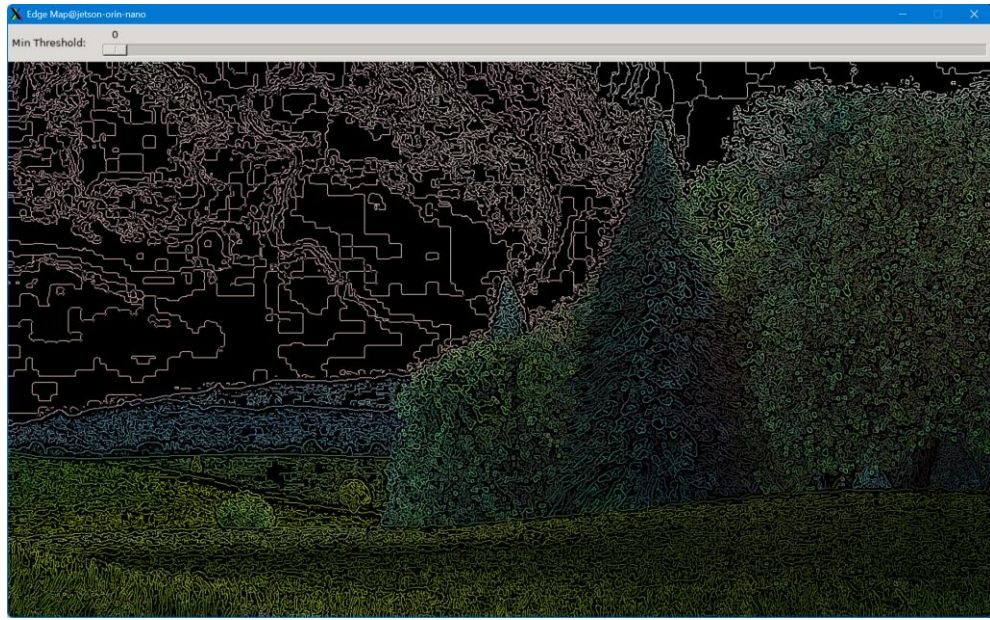


Figure 3.2 Canny Edge Detection on 200th Frame of Big_Buck_Bunny_720p.mp4

4 Compare OpenCV and USB Webcam

Compare the OpenCV camera capture code from previous work and your USB webcam by downloading video capture examples (OpenCV 4.x `simpler-capture4/capture.cpp`, and `simple-capture/` C code built bottom-up with V4L2). Try building, and running each camera capture application and note how each works in good detail. Show a screen dump from each and describe which version you like best and why. Compute your frame rate using time-stamp information (using `posix_clock_gettime` is best, but older code may still use <https://man7.org/linux/man-pages/man2/settimeofday.2.html>) and provide the average frame rate for a time period of 1 minute or more of run time.

The two camera capture examples vary in their complexity and level of abstraction. The OpenCV camera capture code, described in Section 4.1, is very high-level, relying heavily on the OpenCV API. Although the code is much more reduced and easier to understand, the application lacks control that the bottom-up V4L2 code provides. The lower-level code enables manual raw pixel formatting, buffer management, and timestamping that the OpenCV example leaves up to the library. Another key difference between the two applications is that the OpenCV code requires a GUI. The V4L2 example is headless, generating each frame (quantity specified by the user) as a PPM file, without any live user interaction.

Both applications serve distinct use cases. The OpenCV example is built for live monitoring and user interface. The L4V2 example is built for benchmarking, real-time capture, and customization. Personally, I prefer the OpenCV example. I enjoy the live interactive camera display and it is fun, as we have experimented with other examples, such as in Section 2 and Section 3 above, to dynamically change the display within the GUI. However, as an engineer, I understand and appreciate the importance and applicability of the low-level example. This example is more likely to be applied to embedded real-time systems that are resource constrained and configured for specific applications. By developing with more control, the engineer can create better systems.

4.1 `computer_vision_cv4_tested/simpler-capture-4/capture`

This example is a high-level application that is built on OpenCV's API. The use of the API eliminates the hardware interaction, buffer management, and pixel format conversion that executes behind the library class. The application is very readable and follows a simple model. First, open a camera. Then, repeatedly read a frame, display the frame in a named window, and wait for a key press to exit. This example is a blueprint for other more sophisticated examples to build on.

API

`read()`

- Capture an image frame from the VideoCapture device. Reads the image into a Mat object.

imshow()

- Updates the named window display with the Mat object specified by the caller. This is used to display the video camera frame to the user.

waitKey()

- This API call is used to read keyboard input from the user and serves to render graphical displays . It waits the number of milliseconds specified by the caller (or indefinitely if 0 is passed).

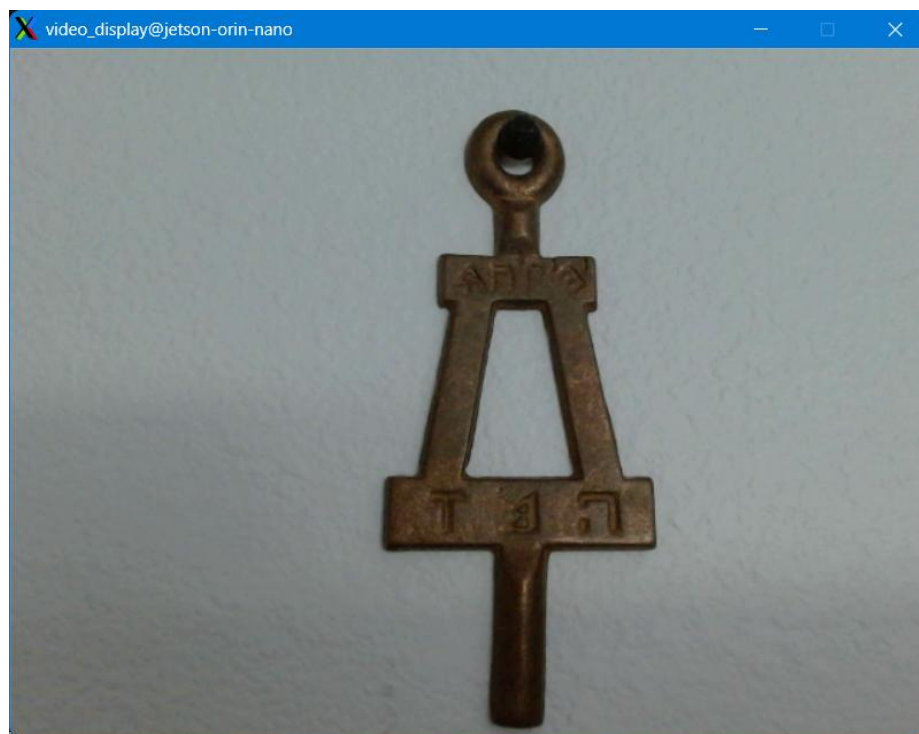


Figure 4.1.3 OpenCV capture application window

While familiarizing with the OpenCV capture application, I observed that the waitKey() API call could be limiting the frame rate of the application. So, I commented out the waitKey() call, remade the application, and noticed that the calculated frame rate was much higher, but there was no GUI generated to display the webcam images. As shown below, the average frame rate over a 60 second period was 14.991fps.

```
mason@jetson-orin-nano:~/ECV_5763/ECV-ECEE-5763-mcgauffin/computer_vision_cv4_tested/simpler-capture-4$ ./capture
[ WARN:0@2.504] global cap_gstreamer.cpp:1728 open OpenCV | GStreamer warning: Cannot query video position: status=0, value=-1, duration=-1
Capture complete: 60.004 seconds elapsed, 900 frames captured
Calculated frame rate: 14.9991 fps
mason@jetson-orin-nano:~/ECV_5763/ECV-ECEE-5763-mcgauffin/computer_vision_cv4_tested/simpler-capture-4$
```

Figure 4.1.0 Terminal Output Frame Rate with no waitKey() API call

Next, I played around with different parameters in the waitKey() API call. They produced varying frame rates as shown in Figure 4.1.2 and Figure 4.1.2, below. With a one (1) millisecond (ms) specified wait time, the calculated frame rate was 4.3343fps. With a ten (10) millisecond (ms) wait time, the frame rate reduced to 4.1330 fps.

```
mason@jetson-orin-nano:~/ECV_5763/ECV-ECEE-5763-mcgaffin/computer_vision_cv4_tested/simpler-capture-4$ ./capture
[ WARN:0@2.499] global cap_gstreamer.cpp:1728 open OpenCV | GStreamer warning: Cannot query video position: status=0, value=-1, duration=-1
Capture complete: 60.218 seconds elapsed, 261 frames captured
Calculated frame rate: 4.3343 fps
```

Figure 4.1.1 Terminal Output Frame Rate with waitKey(1)

```
mason@jetson-orin-nano:~/ECV_5763/ECV-ECEE-5763-mcgaffin/computer_vision_cv4_tested/simpler-capture-4$ ./capture
[ WARN:0@2.494] global cap_gstreamer.cpp:1728 open OpenCV | GStreamer warning: Cannot query video position: status=0, value=-1, duration=-1
Capture complete: 60.247 seconds elapsed, 249 frames captured
Calculated frame rate: 4.1330 fps
```

Figure 4.1.2 Terminal Output Frame Rate with waitKey(10)

4.2 simple-capture/capture

This example is a bare bones application that custom handles pixel formatting, buffer management, and timestamping. The source code is much more complex with multiple helper functions. The application is headless, (differing from the OpenCV example) so each frame is stored as a PPM file with the name corresponding to the frame number of the capture. Despite the added control that this low-level example allows, the overall application process is similar. First, open and configure the camera device. Then, create adequately sized buffers to store the pixel information. Then, read the raw pixel data. Then, convert and store the pixel data as a PPM file.

When building the application, the compiler produced an overflow warning because of the use of strcat() API. To resolve this error, I rewrote the portions applicable to use the safer, snprintf() API. After running the application, I used ImageMagick to view the captured PPM files. I noticed that the first couple of frames, as seen below, are dark and/or discolored. Figure 4.2.0 shows the first frame captured which, as seen, is very dark, almost as if the camera was not ready to capture the image. Figure 4.2.1 shows the third frame captured which appears to have a pink hue to it. This could be the camera adjusting to the light and focusing. Finally, after the first couple frames, the captured images stabilize to what is expected to be seen, similar to Figure 4.2.2. After generating the frames, the application outputs the average frame rate of the execution, 29.8098 fps, as seen in Figure 4.2.3. This application had a much higher frame rate than the OpenCV application. This is probably due to the elimination of the GUI and direct buffer management as opposed to the OpenCV API overhead.

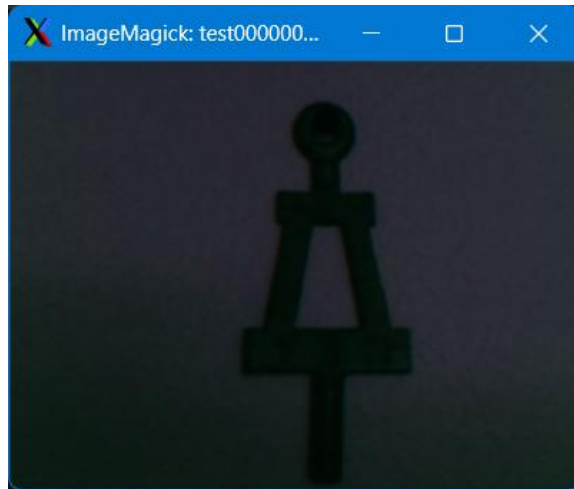


Figure 4.2.0 test00000001.ppm



Figure 4.2.1 test00000003.ppm



Figure 4.2.2 test00000800.ppm

```
frame 1789: Dump YUYV converted to RGB size 153600  
wrote 230400 bytes  
time_error.tv_sec=3377699720530944, time_error.tv_nsec=256  
Capture complete: 60.014 seconds elapsed, 1789 frames captured  
Calculated frame rate: 29.8098 fps
```

Figure 4.2.3 Capture Application Output – Frame Rate

5 Interactive Edge Detection

Using the OpenCV camera capture code from previous work and your USB webcam, create a viewer where you can turn on/off edge detection for Canny and/or Sobel by keystroke (e.g., “c” for Canny, “s” for Sobel, “n” for None). You should be able to build this by combining work on Sobel, Canny, and OpenCV camera capture code you have already completed. This example may however help - capture-transformer which was written for OpenCV 3.x (so update as needed or write your own!). Again, compute the frame average rate for Canny and Sobel test runs using `posix_clock_gettime` time stamping. Add code for this analysis as needed to update and display frame rate updated periodically with OpenCV “put text” on an imshow window or as an update in a syslog (syslog_example). Consider making a slider or other interactive features for thresholds used for Canny and/or Sobel, but this is not required.

I created an interactive edge detection application that allows the user to lean and play with Sobel and Canny edge detection methods. The application allows the user to select if they want to use the live webcam, by passing the appropriate parameter (`‘-l, --live’`). If not, the user can also specify an image file using the appropriate parameters (`‘-f, --file <filename>’`). If the user passes no parameter, the default file, `‘Jeep-Sideview.png’`, is used.

The user can switch between edge detection modes, grayscale, and the unmodified image using the hot keys, `‘R’`, `‘G’`, `‘S’`, and `‘C’`, as shown in Figure 5.1. When in Canny or Sobel mode, the parameter configuration from the trackbars is used in the edge detection method. Using the trackbars, the user can configure the respective parameters for Canny and Sobel edge detection. The trackbars have been modified to meet the parameter ranges and/or restrictions. The Canny Minimum Threshold has a standard range of zero (0) to one-hundred (100). The Sobel kernel size parameter (ksize) is restricted to odd numbers from negative one (-1) to thirty-one (31). The trackbar has been modified to allow input from zero (0) to sixteen (16). The user input, x , has the function, $2x-1$, applied to prevent out-of-bounds input. The Sobel scale and delta parameters have a trackbar range of zero (0) to two-hundred (200). However, they are offset by one-hundred – trackbar input one-hundred (100) correlates to a value of zero (0). This allows for integer parameter values, for scale and delta, between negative one-hundred (-100) and one-hundred (100).

```
mason@jetson-orin-nano:~/ECV_5/63/ex2$ ./interactive_edge -l
The sample uses Canny and Sobel or Scharr OpenCV functions for edge detection

Press 'ESC' to exit program.
Press 'C' for Canny Edge Detection.
Press 'S' for Sobel Edge Detection.
Press 'R' to see Original Image.
Press 'G' to see Grayscale Image
[ WARN:002.274] global cap_gstreamer.cpp:1728 open OpenCV | GStreamer warning: Cannot query video position: status=0, value=-1, duration=-1
[ WARN:009.286] global window.cpp:701 createTrackbar UI/Trackbar(Canny: Min Threshold@Interactive Edge Detector): Using 'value' pointer is unsafe and deprecated. Use NULL as value pointer. To fetch trackbar value setup callback.
[ WARN:009.322] global window.cpp:701 createTrackbar UI/Trackbar(Sobel: ksize@Interactive Edge Detector): Using 'value' pointer is unsafe and deprecated. Use NULL as value pointer. To fetch trackbar value setup callback.
[ WARN:009.323] global window.cpp:701 createTrackbar UI/Trackbar(Sobel: scale@Interactive Edge Detector): Using 'value' pointer is unsafe and deprecated. Use NULL as value pointer. To fetch trackbar value setup callback.
[ WARN:009.323] global window.cpp:701 createTrackbar UI/Trackbar(Sobel: delta@Interactive Edge Detector): Using 'value' pointer is unsafe and deprecated. Use NULL as value pointer. To fetch trackbar value setup callback.
```

Figure 5.1 Interactive Edge Detection Application Terminal Output

When the user runs the application in live mode, using the webcam video, the application displays the live frame rate, calculating the average frame rate over five second periods. After each period, the frame rate is updated on the GUI using the OpenCV `putText()` API and logged using syslog, as seen in Figure 5.2.

```
Jun 14 17:36:38 jetson-orin-nano interactive_edge[1420343]: Interactive Edge Application started using live webcam.
Jun 14 17:36:47 jetson-orin-nano interactive_edge[1420343]: [5s] Calculated frame rate: 4.2371 fps
Jun 14 17:36:55 jetson-orin-nano interactive_edge[1420343]: [10s] Calculated frame rate: 3.0052 fps
Jun 14 17:37:00 jetson-orin-nano interactive_edge[1420343]: [15s] Calculated frame rate: 4.6946 fps
Jun 14 17:37:05 jetson-orin-nano interactive_edge[1420343]: [20s] Calculated frame rate: 2.7751 fps
Jun 14 17:37:11 jetson-orin-nano interactive_edge[1420343]: [25s] Calculated frame rate: 2.5807 fps
Jun 14 17:37:16 jetson-orin-nano interactive_edge[1420343]: [30s] Calculated frame rate: 1.2487 fps
Jun 14 17:37:21 jetson-orin-nano interactive_edge[1420343]: [35s] Calculated frame rate: 1.9770 fps
Jun 14 17:37:27 jetson-orin-nano interactive_edge[1420343]: [40s] Calculated frame rate: 2.2601 fps
Jun 14 17:37:32 jetson-orin-nano interactive_edge[1420343]: [45s] Calculated frame rate: 1.6623 fps
Jun 14 17:37:37 jetson-orin-nano interactive_edge[1420343]: [50s] Calculated frame rate: 5.2374 fps
Jun 14 17:37:48 jetson-orin-nano interactive_edge[1420343]: [55s] Calculated frame rate: 1.1165 fps
Jun 14 17:37:53 jetson-orin-nano interactive_edge[1420343]: [60s] Calculated frame rate: 4.7754 fps
Jun 14 17:37:54 jetson-orin-nano interactive_edge[1420343]: Closing application...
```

Figure 5.2 Interactive Edge Detection Application Syslog Output

The average frame rate for the application was significantly lower than I expected. This is most likely because I was on the road for two weeks during this assignment. I was still able to use my Jetson Orin Nano setup at home. After some security configurations to the device (no passwords, etc), I exposed a port on my home Wi-Fi router that allows me to connect to the device with an SSH Key from anywhere with an internet connection. However, SSH can only do so much for high bandwidth data such as video. I predict this limited my frame rate and added significant latency. Also, because of the large amount of data being transferred over long distances, I occasionally ran into the problem shown below in Figure 5.3. This error was likely a bit-flip that caused the SSH message authentication code to fail.

```
ssh_dispatch_run_fatal: Connection to [REDACTED] port 2223: message authentication code incorrect
nmc3334@mcgaff:~$
```

Figure 5.3 SSH Message Authentication Error

As mentioned in the section prior, my average frame rate was typically lower than expected. I was mainly on a friend's Wi-Fi that was being used by multiple devices or a public Wi-Fi network at a café or similar. However, there were times when the frame rate improved. I was able to capture some of these moments as seen in the Figures below.

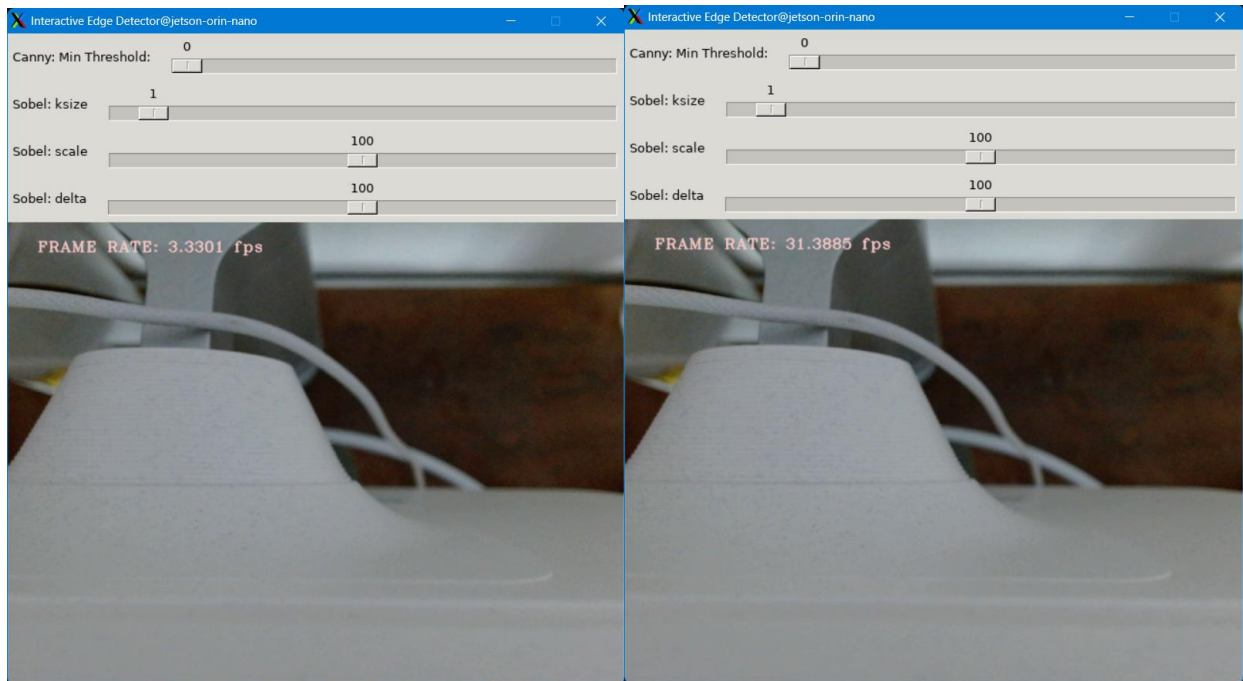


Figure 5.4 & 5.5 Regular Mode

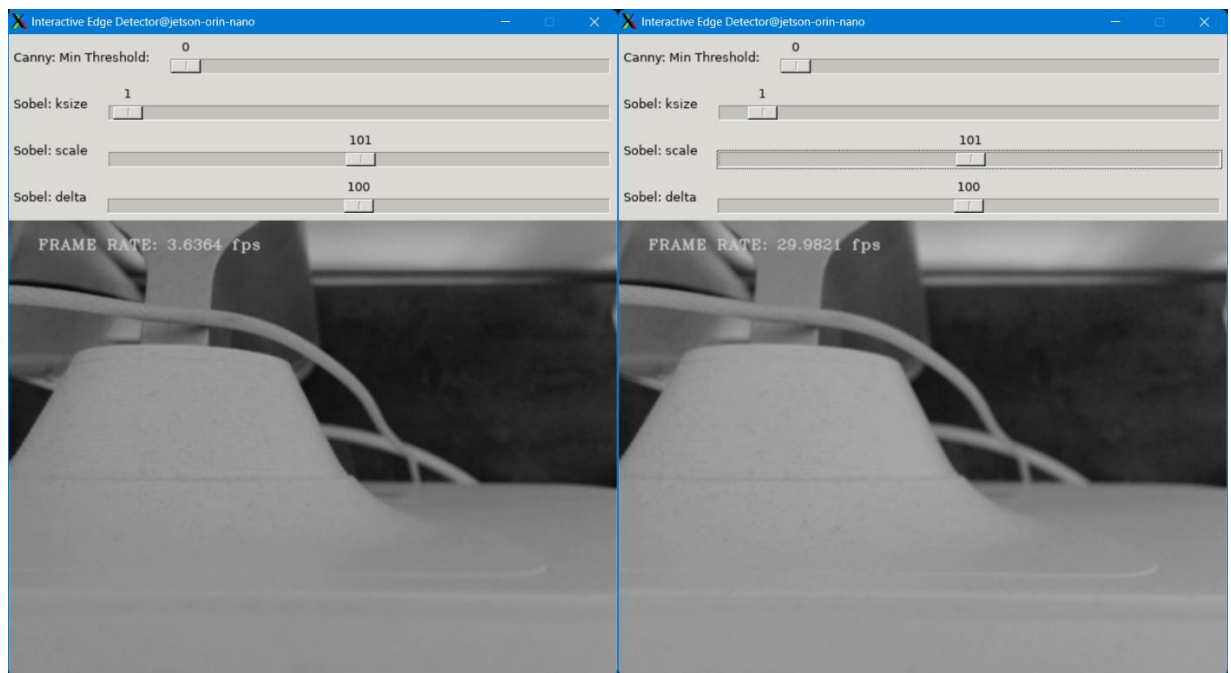


Figure 5.6 & 5.7 Grayscale Mode

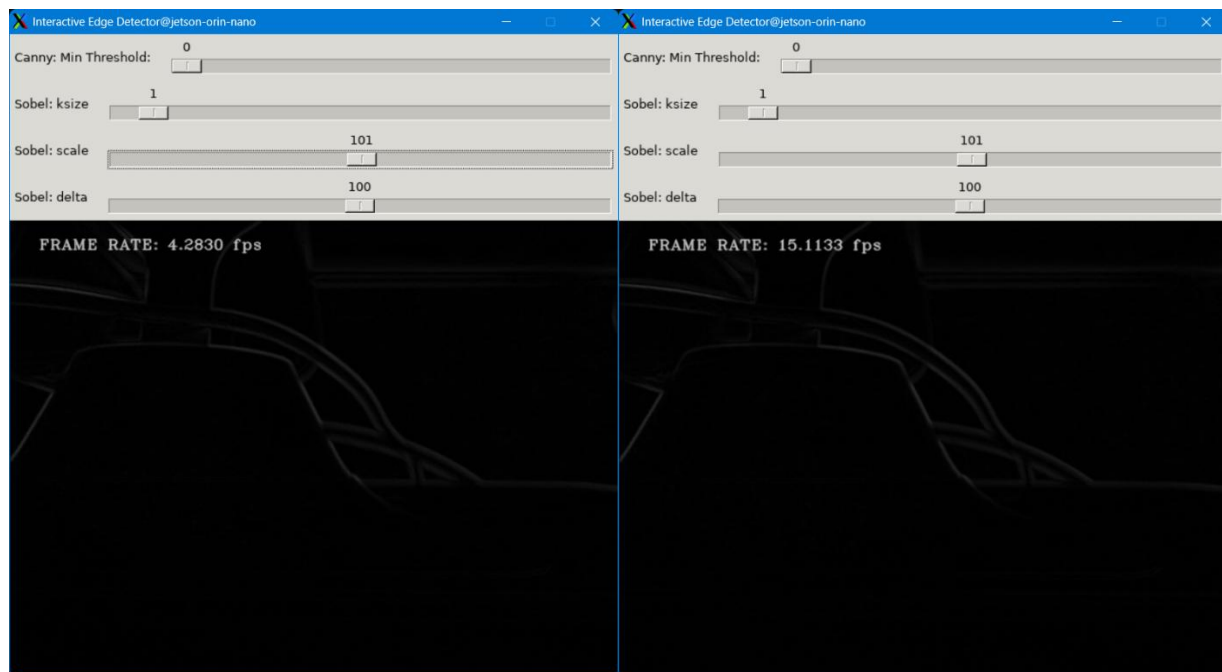


Figure 5.8 & 5.9 Sobel Mode

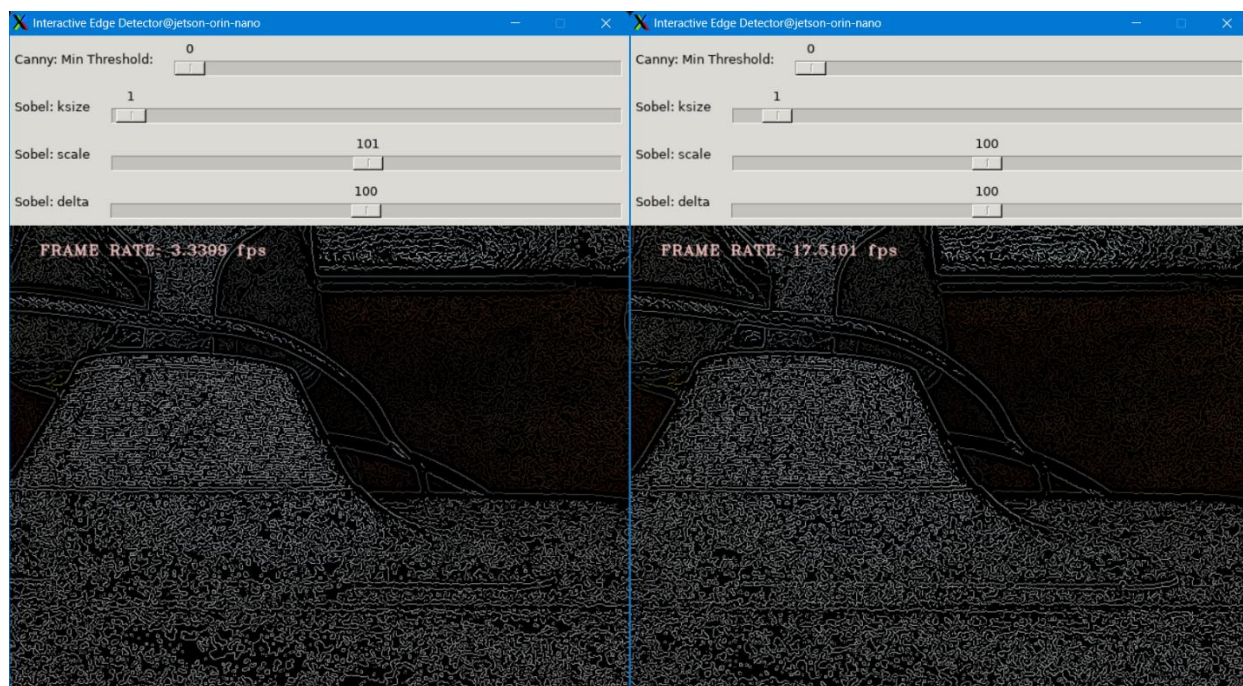


Figure 5.10 & 5.11 Canny Mode

6 References

[1] FFmpeg Developers, "FFmpeg Frequently Asked Questions," *FFmpeg Official Documentation*, 2026. [Online]. Available: <https://ffmpeg.org/faq.html>

[2] OpenCV, "OpenCV 4.8.0 Documentation." [Online]. Available: <https://docs.opencv.org/4.8.0/index.html>.

[3] OpenCV, "Cascade Classifier Tutorial." [Online]. Available: https://docs.opencv.org/4.8.0/db/d28/tutorial_cascade_classifier.html.

7 Appendix

GitHub Repo: https://github.com/mamc3334/ECV_5763/tree/ex2

Pull Request: https://github.com/mamc3334/ECV_5763/pull/3

Forked ECV Repo: <https://github.com/mamc3334/ECV-ECEE-5763-mcgaffin/tree/ex2>

Pull Request: <https://github.com/mamc3334/ECV-ECEE-5763-mcgaffin/pull/2>

Build Environment

OS: Ubuntu 22.04.5 LTS

Kernel: 5.15.148-tegra

Architecture: aarch64

gcc/g++ 11.4.0

GNU Make 4.3

GNU gdb 12.1

GLIBC 2.35

util-linux 2.37.2

system 249